



END-TO-END ETL PIPELINE USING AIRFLOW & SPARK (MYSQL-CSV)

TECHNICAL TEST

FULL CODE IN GITHUB: <https://github.com/fauziah61/End-To-End-Data-Pipeline-Using-Airflow-Docker-MySQL-CSV/tree/main>

NAME OF PROJECT:
TECHNICAL TEST

CREATED BY:
FAUZIAH HABIBAH

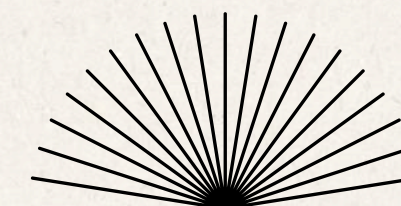


Table of Contents

01	About The Project/Task 3
02	Tools
03	Docker Configuration/Task Optional
03	Script/Task 1 & 2
04	The Results

01 Project ini berfokus pada pembuatan data pipeline menggunakan airflow untuk Perusahaan retail otomotif yg sedang membangun pusat data dan migrasi data dari yang tadinya banyak menggunakan excel. Datawarehouse yang digunakan adalah MySQL.

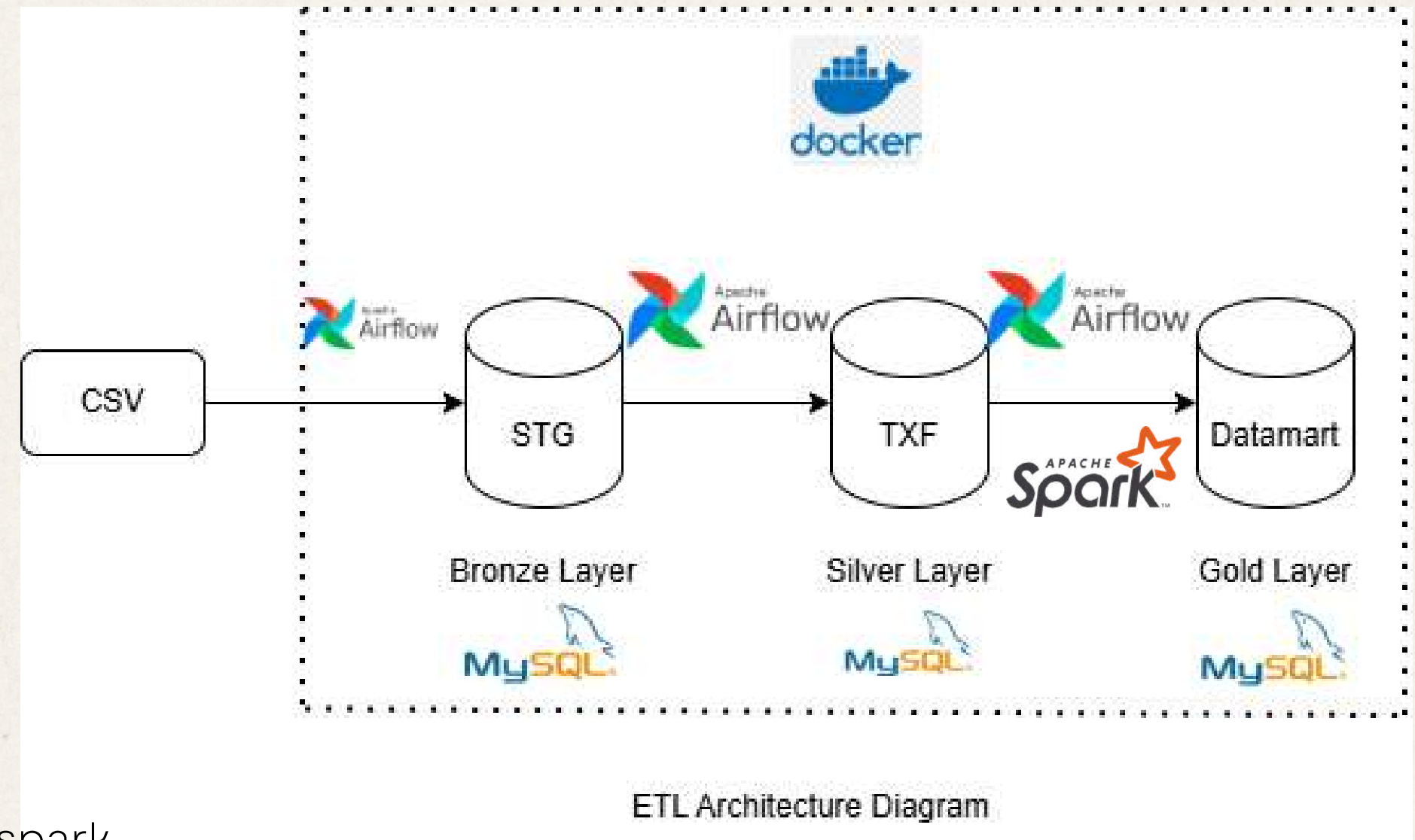
02 Di MySQL, sudah ada table customers_raw, sales_raw, dan after_sales_raw.

Masih ada source data dari filesharing yaitu customer_addresses_yyyymmdd.csv.

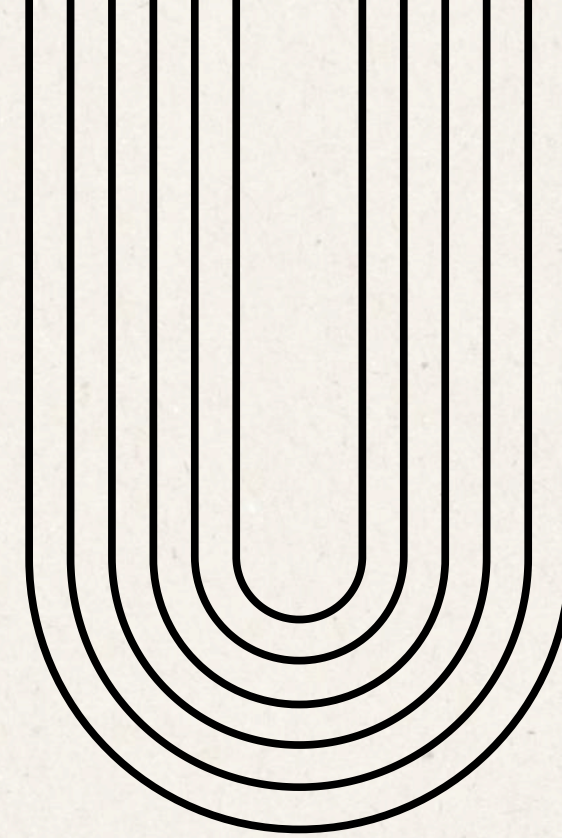
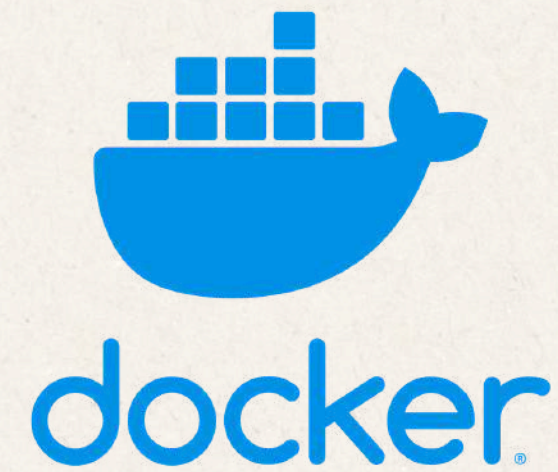
- 03**
- Table bronze, silver, dan gold berada di 1 database yang sama, bisa dipisah menyesuaikan bisnis case apabila data sudah terlalu besar. dibuat bertahap agar dapat ditrace apabila ada error
 - Table bronze layer/stg/staging berisi tabel dengan type data varchar semua
 - Table silver layer/txf berisi tabel dengan type data sesuai dengan kolom (contoh date, int, dst) yang sudah dibersihkan dan distandarisasi
 - Table gold layer/datamart berisi datamart dengan type data sesuai dengan table txf

- 04**
- Job airflow dijalankan menggunakan docker sebagai kontenarisasi, job data mart dijalankan menggunakan spark
 - Job airflow dijalankan tiap hari, tabel stg di jam 5, tabel txf di jam 8, dan tabel datamart di jam 10, jam bisa diubah menyesuaikan kebutuhan perusahaan

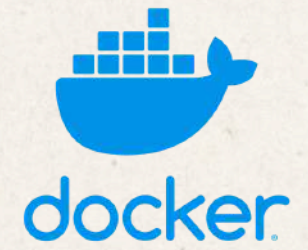
About The Project/Task



Tools



Docker Configuration



Run Docker & Airflow



**Docker
compose.yaml**



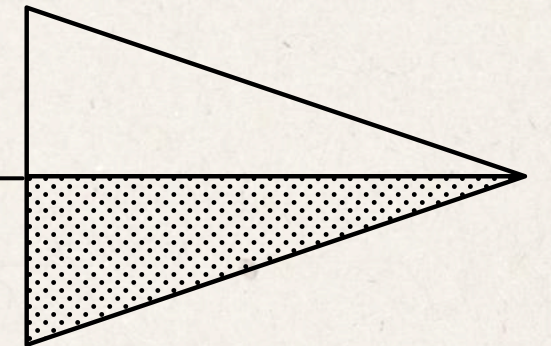
Dockerfile
Untuk menjalankan
requirements.txt



Requirements.txt



**Put Python Scripts in
dag folder**



Docker Configuration

Dockerfile

Untuk menjalankan requirements.txt

Requirements.txt

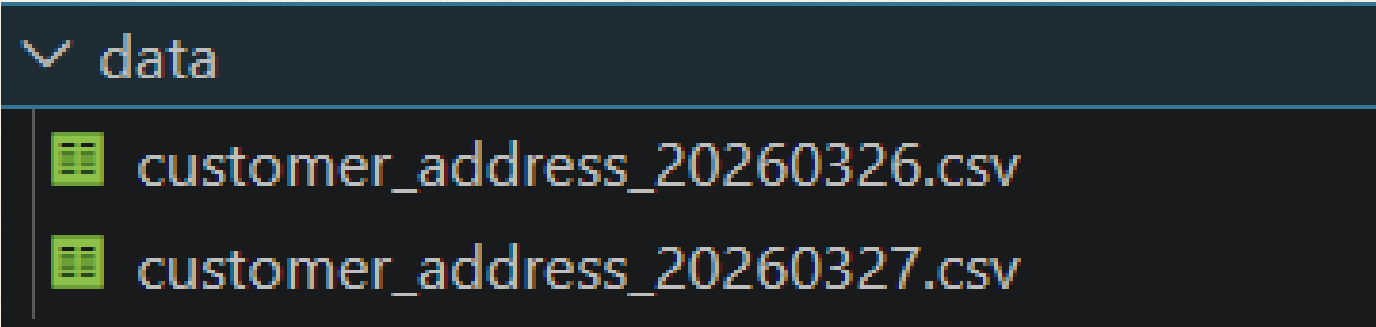
```
dockerfile > FROM
1 FROM apache/airflow:3.1.8
2 ADD requirements.txt .
3 RUN pip install apache-airflow==${AIRFLOW_VERSION} -r requirements.txt
```

```
requirements.txt
1 apache-airflow==3.1.8
2 apache-airflow-providers-mysql
3 pandas
4 sqlalchemy
5 mysqlclient
6 psycopg2-binary==2.9.9
7 time
8
```

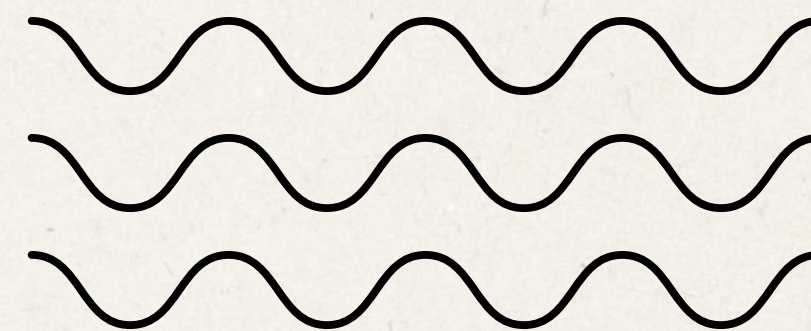
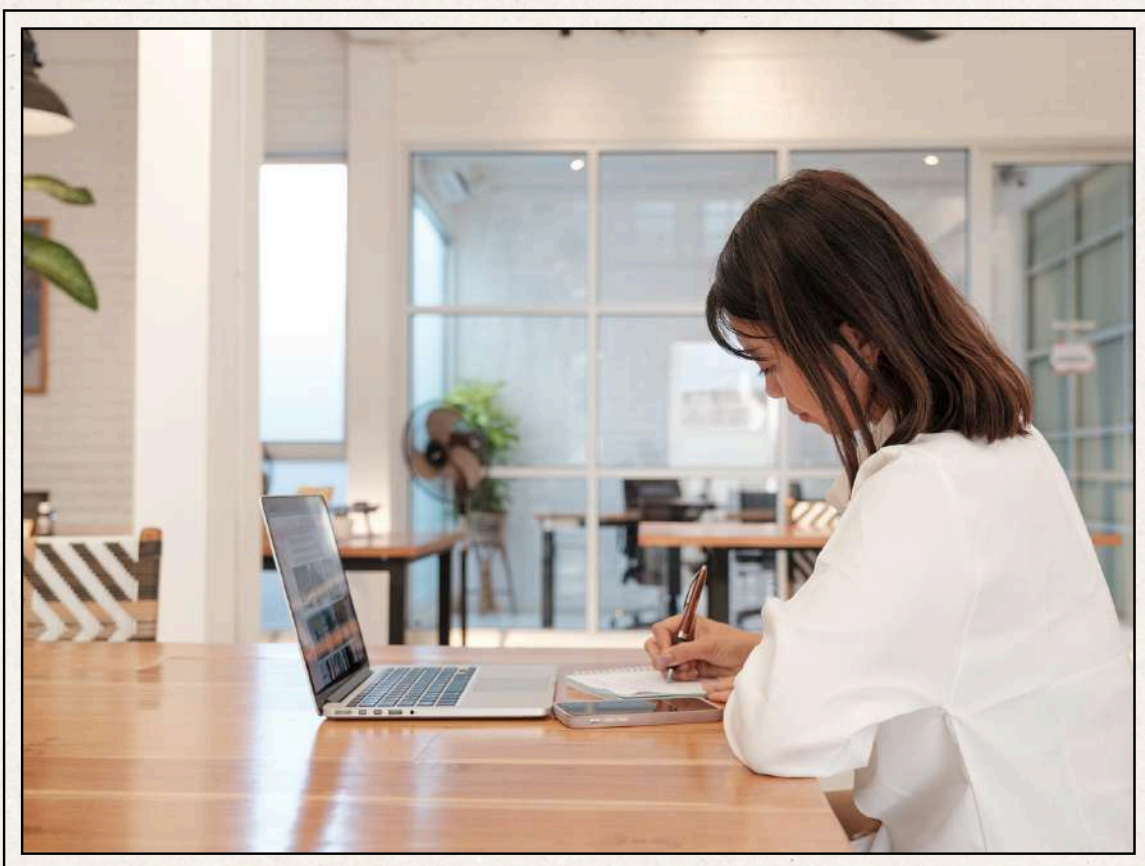
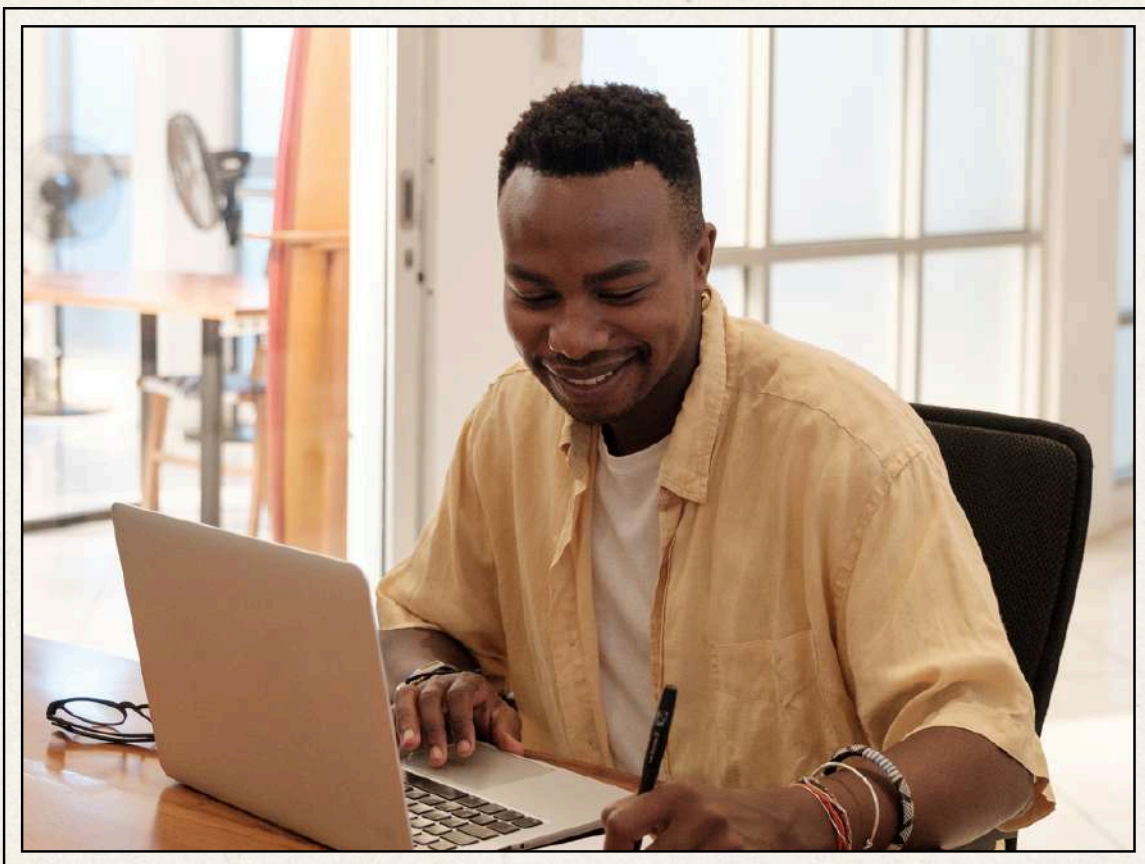
Connection to Database & Landing File

Connection ID ▾	Connection Type ▾	Description ▾	Host ▾	Port ▾
mysql-localhost	mysql	localhost	host.docker.internal	3306

Dimana database yang digunakan adalah mysql lokal



Landing file ada di local dengan path: D:/Git/Repository/End-To-End-Data-Pipeline-Using-Airflow-Docker-MySQL-CSV/data



SCRIPTS, TASKS & THE RESULTS (Task 1 & 2)

Create Dummy Data First

Sebelum script airflow dijalankan, insert data sampel terlebih dahulu dan jalankan script generate dummy data (hanya dijalankan 1x) karena menyesuaikan dengan study case dimana table customers_raw, sales_raw, dan after_sales_raw sudah exist di database & file customer_addresses_yyyymmdd.csv keluar harian di landing.

--DATA SAMPLE DI INPUT MANUAL (DIJALANKAN 1X AJA), SEBELUM DIAMBIL DATA DUMMY

```
INSERT INTO `mysql-dwh-dev`.stg_customers_raw
VALUES
```

```
(1, 'Antonio', '1998-08-04', '2025-03-01 14:24:40.012'),
(2, 'Brandon', '2001-04-21', '2025-03-02 08:12:54.003'),
(3, 'Charlie', '1980/11/15', '2025-03-02 11:20:02.391'),
(4, 'Dominikus', '14/01/1995', '2025-03-03 09:50:41.852'),
(5, 'Erik', '1900-01-01', '2025-03-03 17:22:03.198'),
(6, 'PT Black Bird', NULL, '2025-03-04 12:52:16.122');
```

```
INSERT INTO `mysql-dwh-dev`.STG_SALES_RAW
VALUES
```

```
('JIS8135SAD', 1, 'RAIZA', '2025-03-01', '350.000.000', '2025-03-01 14:24:40.012'),
('MAS8160POE', 3, 'RANGGO', '2025-05-19', '430.000.000', '2025-05-19 14:29:21.003'),
('JLK1368KDE', 4, 'INNAVO', '2025-05-22', '600.000.000', '2025-05-22 16:10:28.120'),
('JLK1869KDF', 6, 'VELOS', '2025-08-02', '390.000.000', '2025-08-02 14:04:31.021'),
('JLK1962KOP', 6, 'VELOS', '2025-08-02', '390.000.000', '2025-08-02 15:21:04.201')
;
```

```
INSERT INTO `mysql-dwh-dev`.STG_AFTER_SALES_RAW
VALUES
```

```
('T124-kgu1', 'MAS8160POE', 3, 'RANGGO', '2025-07-11', 'BP', '2025-07-11 09:24:40.012'),
('T560-jga1', 'JLK1368KDE', 4, 'INNAVO', '2025-08-04', 'PM', '2025-08-04 10:12:54.003'),
('T521-oai8', 'POI1059IIK', 5, 'RAIZA', '2026-09-10', 'GR', '2026-09-10 12:45:02.391');
```

```
INSERT INTO `mysql-dwh-dev`.STG_CUSTOMER_ADDRESS
VALUES
```

```
(1,1,'Jalan Mawar V', 'RT 1/RW 2', 'Bekasi', 'Jawa Barat', '2026-03-01 14:24:40.012'),
(2,3,'Jl Ababil Indah', 'Tangerang Selatan', 'Jawa Barat', '2026-03-01 14:24:40.012'),
(3,4,'Jl. Kemang Raya 1 No 3', 'JAKARTA PUSAT', 'DKI JAKARTA', '2026-03-01 14:24:40.012'),
(4,6,'Astra Tower Jalan Yos Sudarso 12', 'Jakarta Utara', 'DKI Jakarta', '2026-03-01 14:24:40.012');
```

01

Insert data sampel, di database MySQL

detailed script ada di:

<https://github.com/fauziah61/End-To-End-Data-Pipeline-Using-Airflow-Docker-MySQL-CSV/blob/main/Insert-data-sample-manual.sql>

02

Generate data dummy, Library python yang

digunakan: Random, Faker, String, OS,

Mysql.connector

Inject dummy data as existing table in db

Table existing hasil dummy data di-inject ke tabel staging stg_customers_raw, stg_sales_raw, dan stg_after_sales_raw.

mysql-localhost localhost:3306

Databases

mysql-dwh-dev

Tables

mart_cust_service_prio_datamart

mart_sales_datamart16K

stg_after_sales_raw128K

stg_customer_address144K

stg_customers_raw96K

Columns

AZ ID (varchar(255))

AZ NAME (varchar(255))

AZ DOB (varchar(50))

AZ CREATED_AT (varchar(50))

	AZ ID	AZ NAME	AZ DOB	AZ CREATED_AT
1	1	Antonio	1998-08-04	2025-03-01 14:24:40.012
2	10	Julie	12/09/1980	2026-03-04 13:46:25.242
3	100	Wang Inc	[NULL]	2026-01-30 17:50:26.136
4	1000	Jon	13/12/2004	2026-01-01 18:55:58.582
5	101	Catherine	[NULL]	2026-01-27 05:28:01.237
6	102	Daniel	2008/07/11	2026-03-11 12:59:36.404
7	103	Tina	2005/09/21	2026-01-04 06:36:47.485
8	104	Manuel	1942-06-15	2026-03-08 16:16:59.407
9	105	Jason	14/01/2006	2026-02-17 18:34:50.204
10	106	Oscar	10/02/1952	2026-01-10 21:35:05.672
11	107	Katherine	[NULL]	2026-02-02 01:20:37.182

mysql-localhost localhost:3306

Databases

mysql-dwh-dev

Tables

mart_cust_service_prio_datamart

mart_sales_datamart16K

stg_after_sales_raw128K

stg_customer_address144K

stg_customers_raw96K

stg_sales_raw

Columns

AZ VIN (varchar(25))

AZ CUSTOMER_ID (varchar(255))

AZ MODEL (varchar(50))

AZ INVOICE_DATE (varchar(50))

AZ PRICE (varchar(255))

AZ CREATED_AT (varchar(50))

	AZ VIN	AZ CUSTOMER_ID	AZ MODEL	AZ INVOICE_DATE	AZ PRICE	AZ CREATED_AT
1	5SV5H0EJWQ	17	INNAVO	2025-02-18	600.000.000	2025-02-18 18:26:07.000
2	AX47YKVAXB	869	RAIZA	2025-01-27	350.000.000	2025-01-27 11:21:52.000
3	Y0GFYJL31T	848	RANGGO	2025-03-02	430.000.000	2025-03-02 20:51:14.000
4	EVX5V9151W	471	RANGGO	2025-01-12	430.000.000	2025-01-12 04:27:40.000
5	8MYWTURKY7	687	INNAVO	2025-01-15	600.000.000	2025-01-15 18:09:09.000
6	ES2JFL3W0I	111	RAIZA	2025-02-08	350.000.000	2025-02-08 12:40:33.000
7	7WTCQBMD4	694	RAIZA	2025-03-17	350.000.000	2025-03-17 08:14:55.000
8	Y5GRME8U0Z	858	RANGGO	2025-02-11	430.000.000	2025-02-11 08:55:28.000
9	TRX3BCIOD2	437	VELOS	2025-03-28	390.000.000	2025-03-28 14:36:52.000
10	CTF2OBTUSZ	945	INNAVO	2025-02-19	600.000.000	2025-02-19 12:26:08.000
11	67OB447M4J	319	VELOS	2025-03-02	390.000.000	2025-03-02 07:28:45.000
12	7AMWN5LE3N	816	VELOS	2025-01-29	390.000.000	2025-01-29 13:00:53.000
13	YXZQFCHIAH	889	VELOS	2025-03-05	390.000.000	2025-03-05 20:23:24.000
14	ME7DFVNDIX	373	VELOS	2025-02-11	390.000.000	2025-02-11 13:21:30.000

AZ service_ticket

AZ vin

AZ customer_id

AZ model

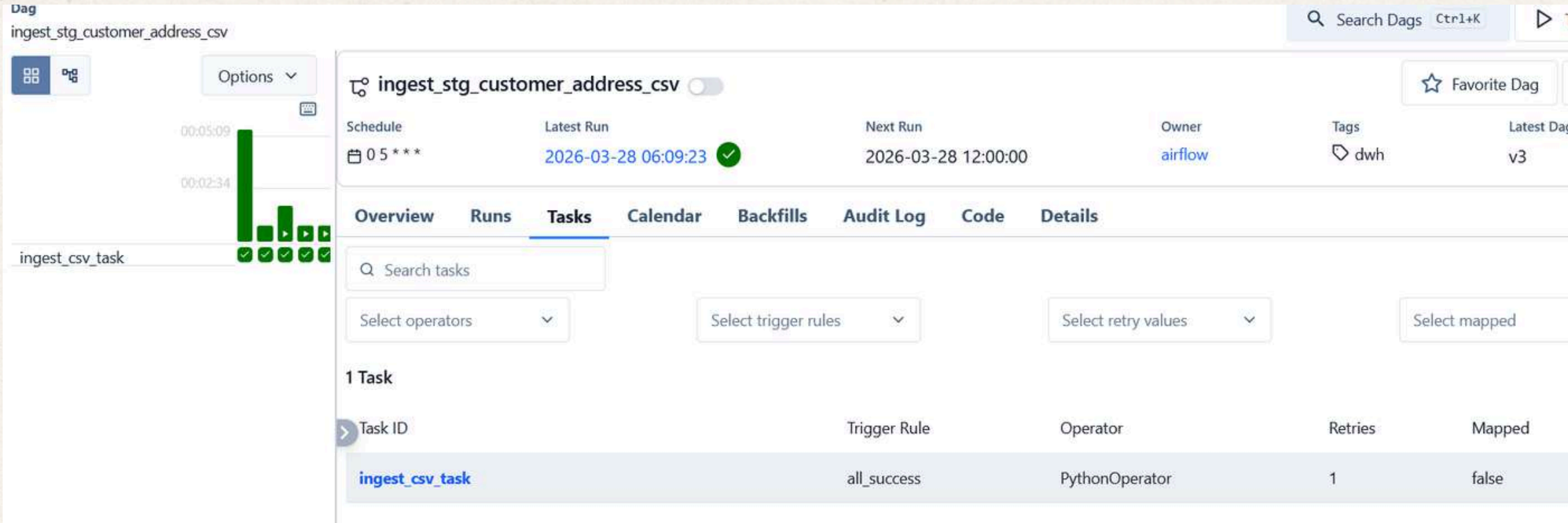
service_date

AZ service_type

created_at

1	T103-aem4	J88GVQUUX3	143	VELOS	2025-10-19	BP	2025-10-19 21:01:28.240
2	T105-fez8	I4I7X7JNQ9	810	INNAVO	2026-05-23	PM	2026-05-23 13:38:59.065
3	T105-uwo8	RYI2DSX37B	494	RAIZA	2025-11-30	BP	2025-11-30 15:09:36.345
4	T108-giv9	GKL4JPPSBA	5	RANGGO	2025-09-02	GR	2025-09-02 07:40:42.678
5	T108-zda4	W99VFFB2RN	17	VELOS	2026-06-10	BP	2026-06-10 18:40:55.794
6	T109-ziq4	JDQIADZVMG	318	RAIZA	2025-08-08	BP	2025-08-08 07:14:50.384
7	T112-cyu6	DRJVAP0ATD	949	RANGGO	2026-11-05	BP	2026-11-05 22:20:20.999
8	T112-ebw8	HIRAC0VGYQ	875	RANGGO	2025-03-03	GR	2025-03-03 22:46:17.097
9	T113-abm1	A0G78U1E97	70	RAIZA	2025-10-22	BP	2025-10-22 16:04:37.390
10	T113-rgk5	GDDNMAC5Q6	599	VELOS	2025-07-03	PM	2025-07-03 20:33:45.328
11	T115-fen8	3XOSN7CHWS	773	VELOS	2026-12-01	GR	2026-12-01 16:31:41.423
12	T115-ukb9	GSHHWNJPPH	612	VELOS	2026-11-20	GR	2026-11-20 12:53:15.564
13	T115-wjv3	41X6XJRZ5E	767	RAIZA	2026-06-05	PM	2026-06-05 21:30:56.687
14	T116-olt9	5G5P6262V9	53	INNAVO	2025-10-05	GR	2025-10-05 09:18:22.463
15	T116-rkh3	92VN7XV9I3	933	INNAVO	2026-05-06	PM	2026-05-06 17:53:32.355
16	T117-lzg8	QK838I8F06	751	INNAVO	2026-04-13	PM	2026-04-13 17:54:30.991
17	T117-atp1	YP65NX901W	255	RANGGO	2025-08-28	GR	2025-08-28 02:49:44.667

Task 01: Datalanding



Dijalankan menggunakan airflow 3.1.8 dengan docker sebagai kontenarisasi

PY Script 01: Datalanding

```
from airflow import DAG
# from airflow.operators.python import PythonOperator -> muncul warning karena
from airflow.providers.standard.operators.python import PythonOperator
from airflow.providers.mysql.hooks.mysql import MySqlHook
from datetime import datetime, timedelta
# from airflow.datasets import Dataset
import os
import pandas as pd
import time

# CONFIG
BASE_PATH = "/opt/airflow/data"
TABLE_NAME = "stg_customer_address"
MYSQL_CONN_ID = 'mysql-localhost'

# # DATASET (event)
# customer_address_dataset = Dataset("stg_customer_address")

default_args = {
    'owner': 'airflow',
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

def get_file_path(ds):
    file_date = ds.replace('-', '')
    return os.path.join(BASE_PATH, f"customer_address_{file_date}.csv")

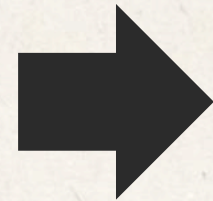
def wait_for_stable_file(file_path, wait_time=5, check_interval=1): # WFF: Wait for File
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"File tidak ditemukan: {file_path}")

    last_size = -1
    stable_time = 0

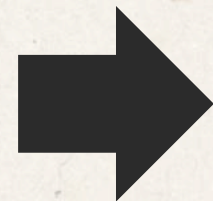
    while True:
        current_size = os.path.getsize(file_path)

        if current_size == last_size:
            stable_time += check_interval
        else:
            stable_time = 0
            last_size = current_size

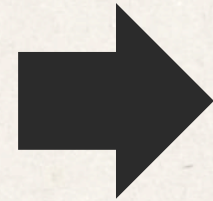
        if stable_time >= wait_time:
```



Library yang digunakan



Define path & connection



Menggunakan wait for file/file checker size, apabila dalam 5 detik file sudah tidak bertambah sizenya, baru diambil, untuk menghindari data tidak lengkap terinject ke table db, apabila file masih bertambah/write maka sleep dan tunggu

PY Script 01: Datalanding

```
> task_ingest_stg_customer_address.py > ingest_csv
def wait_for_stable_file(file_path, wait_time=5, check_interval=1): # WFF
    break
    time.sleep(check_interval)

def ingest_csv(**context):
    # ds = context['ds']
    # file_path = get_file_path(ds)
    input_date = context.get("params", {}).get("input_date")
    # Cara menjalankan input data manual -> di kolom json
    # {
    # "input_date": "2026-03-26"
    # }

    ds = input_date if input_date else context['ds'] # Run by scheduler

    file_path = get_file_path(ds)

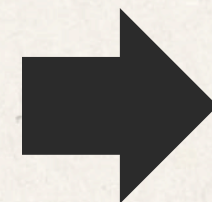
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"File tidak ditemukan: {file_path}")

    # Tunggu file selesai ditulis (size stabil)
    wait_for_stable_file(file_path, wait_time=5)

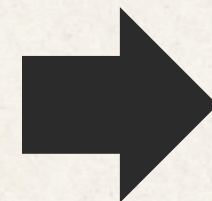
    df = pd.read_csv(file_path, sep='\\|@~', engine='python')

    # Raise error jika koneksi gagal
    try:
        mysql_hook = MySqlHook(mysql_conn_id=MYSQL_CONN_ID)
        conn = mysql_hook.get_conn()
        conn.ping()
        cursor = conn.cursor()
    except Exception as e:
        raise ConnectionError(f"Gagal konek ke MySQL: {str(e)}")

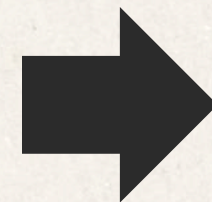
    # DDL
    create_table_sql = f"""
    CREATE TABLE IF NOT EXISTS `{TABLE_NAME}` (
        id VARCHAR(255),
        customer_id VARCHAR(255) NOT NULL,
        address VARCHAR(255),
        city VARCHAR(50),
        province VARCHAR(25),
        created_at VARCHAR(50),
```



Setting parameter input_date & context['ds'] supaya job-nya bisa dijalankan manual (menggunakan json c/o: {"input_date":"2025-01-01"}) atau by scheduler



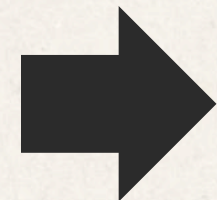
Raise exception error jika file tidak ditemukan



Raise exception error jika gagal terkoneksi ke database

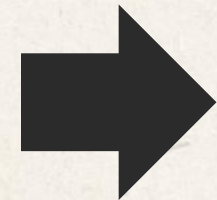
PY Script 01: Datalanding

```
task_ingest_stg_customer_address.py > ingest_csv
def ingest_csv(**context):
    # DDL
    create_table_sql = f"""
    CREATE TABLE IF NOT EXISTS `{TABLE_NAME}` (
        id VARCHAR(255),
        customer_id VARCHAR(255) NOT NULL,
        address VARCHAR(255),
        city VARCHAR(50),
        province VARCHAR(25),
        created_at VARCHAR(50),
        load_timestamp TIMESTAMP(6) DEFAULT CURRENT_TIMESTAMP(6),
        PRIMARY KEY (customer_id)
    );
    """
    cursor.execute(create_table_sql)
    conn.commit()
    # UPSERT SCD Type 1 Karena tidak ada requirement untuk menyimpan history,
    # jadi cukup update data lama dengan data baru jika terjadi duplikasi customer_id
    insert_sql = f"""
    INSERT INTO `{TABLE_NAME}` (
        id, customer_id, address, city, province, created_at
    ) VALUES (%s, %s, %s, %s, %s, %s)
    ON DUPLICATE KEY UPDATE
        address = VALUES(address),
        city = VALUES(city),
        province = VALUES(province),
        created_at = VALUES(created_at),
        load_timestamp = CURRENT_TIMESTAMP(6);
    """
    data = list(df.itertuples(index=False, name=None))
    try:
        for row in data:
            cursor.execute(insert_sql, row)
            conn.commit()
    except Exception as e:
        conn.rollback()
        raise RuntimeError(f"Gagal insert/update data: {e}")
    finally:
        cursor.close()
        conn.close()
    print(f"{len(df)} rows processed (insert/update)")
```

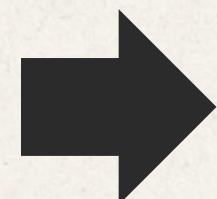


- DDL If not exist, menggunakan load_timestamp sbg control column, disimpan sebagai table stg/staging/bronze layer dimana semua tipe data varchar
- Insert data

menggunakan type load update-insert (SCD Type 1) Karena tidak ada requirement untuk menyimpan history, jadi cukup update data lama dengan data baru jika terjadi duplikasi customer_id



Raise exception error rollback apabila gagal insert



Set dag id, tag & scheduler tiap hari jam 5 pagi

```
with DAG(
    dag_id='ingest_stg_customer_address_csv',
    default_args=default_args,
    schedule='0 5 * * *', # jam 05:00
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=['dwh'],
) as dag:

    ingest_task = PythonOperator(
        task_id='ingest_csv_task',
        python_callable=ingest_csv
    )
```

Task 01 Datalanding: Staging Customer Address Result

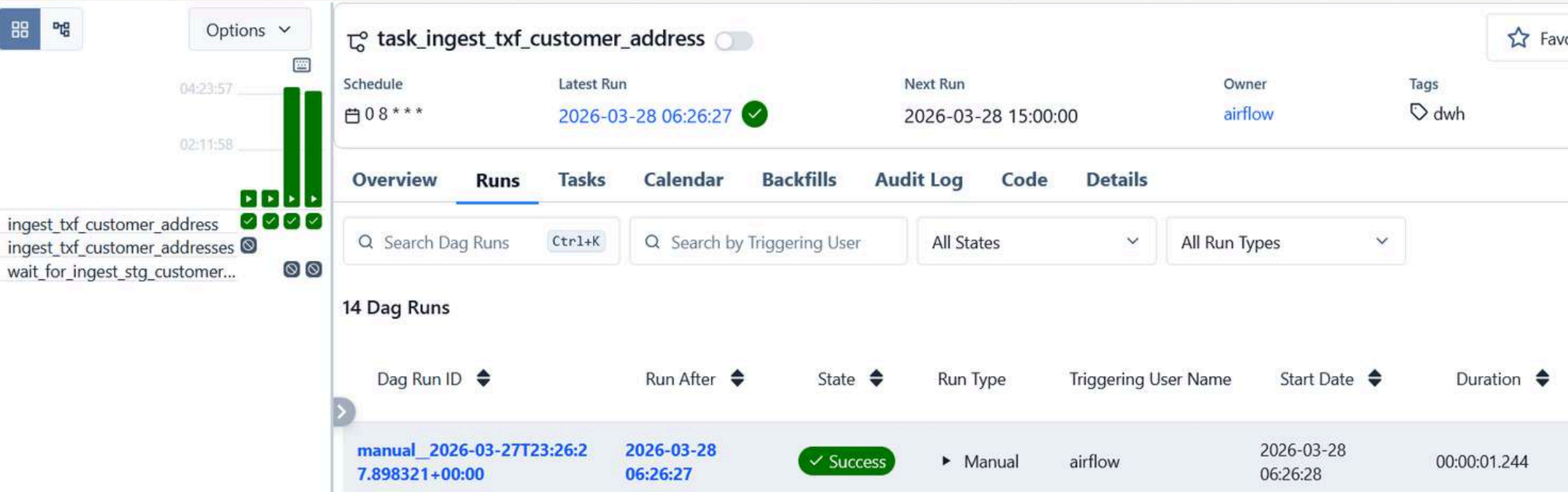
The screenshot displays a database management interface. On the left, a tree view shows the database structure: 'localhost - localhost:3306' contains a 'covid_19' database, which has a 'covid19_jabar' table with 64K records. The table's columns are listed, including '123 CLOSECONTACT (bigint)', '123 CONFIRMATION (bigint)', '123 PROBABLE (bigint)', '123 SUSPECT (bigint)', '123 closecontact_dikarantina (bigint)', '123 closecontact_discarded (bigint)', '123 closecontact_meninggal (bigint)', '123 confirmation_meninggal (bigint)', '123 confirmation_sembuh (bigint)', 'ABC kode_kab (text)', 'ABC kode_prov (text)', 'ABC nama_kab (text)', 'ABC nama_prov (text)', '123 probable_dilolasi (bigint)', '123 probable_discarded (bigint)', '123 probable_meninggal (bigint)', '123 suspect_dilolasi (bigint)', '123 suspect_discarded (bigint)', '123 suspect_meninggal (bigint)', and 'ABC tanggal (text)'. On the right, the 'Grid' view shows the first 23 records of the '123 CLOSECONTACT' table. The columns are: '123 CLOSECONTACT', '123 CONFIRMATION', '123 PROBABLE', '123 SUSPECT', '123 closecontact_dikarantina', and '123 closecontact_discarded'. The data is as follows:

	123 CLOSECONTACT	123 CONFIRMATION	123 PROBABLE	123 SUSPECT	123 closecontact_dikarantina	123 closecontact_discarded
1	274	0	26	2,210	0	
2	534	0	7	776	72	
3	2,127	0	33	5,536	135	
4	0	0	163	0	0	
5	1,295	0	3	2,075	3	
6	0	0	0	0	0	
7	442	0	1	300	187	
8	2,382	0	0	2,888	132	
9	235	0	73	1,640	35	
10	1,437	0	57	6,100	118	
11	471	0	1	1,990	4	
12	420	0	7	671	260	
13	537	0	0	573	0	
14	532	0	1	691	39	
15	455	0	0	5,788	50	
16	332	0	25	4,718	0	
17	125	0	4	1,043	0	
18	0	0	0	0	0	
19	3,553	0	0	6,780	16	
20	187	0	0	586	0	
21	0	0	0	0	0	
22	892	0	1	2,292	68	
23	3,926	0	0	856	318	

Task 02: Silver Layer/txf (task 2.a cleansing data)

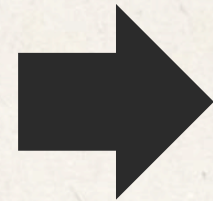
1. Table customer_address

Table **stg_customer_address** di cleansing dan diubah tipe datanya dari yang sebelumnya varchar jadi menyesuaikan dengan kolom yang ada (contoh date, int, dst) dan di-inject ke table table **txf_customer_address** dengan job airflow yang jalan daily tiap jam

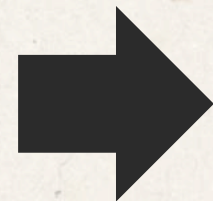


PY Script 02: TXF_CUSTOMER_ADDRESS (CLEANSING & INJECT FROM TABLE STG_CUSTOMER_ADDRESS TO TABLE TXF_CUSTOMER_ADDRESS)

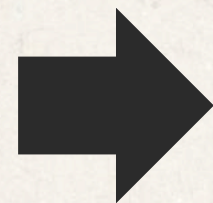
```
js > task_ingest_txf_customer_address.py > ...
1 from time import time
2 from airflow import DAG
3 from airflow.decorators import task
4 from airflow.operators.python import PythonOperator
5 from airflow.providers.mysql.hooks.mysql import MySqlHook
6 # from airflow.sensors.external_task import ExternalTaskSensor
7 # from airflow.datasets import Dataset
8 from datetime import datetime, timedelta
9
10 MYSQL_CONN_ID = 'mysql-localhost'
11
12 # # SAME DATASET
13 # customer_address_dataset = Dataset("stg_customer_address")
14
15 default_args = {
16     'owner': 'airflow',
17     'retries': 0,
18     'retry_delay': timedelta(minutes=5),
19 }
20
21 @task
22 def ingest_txf_customer_address(**context):
23     # Ambil tanggal Airflow (manual / scheduler)
24     input_date = context.get("params", {}).get("input_date")
25     # ds = input_date if input_date else context['logical_date'].strftime('%Y-%m-%d')
26     ds = input_date if input_date else context['ds'] # YYYY-MM-DD
27     try:
28         mysql_hook = MySqlHook(mysql_conn_id=MYSQL_CONN_ID)
29         conn = mysql_hook.get_conn()
30         # conn.autocommit = True
31         conn.ping()
32         cursor = conn.cursor()
33     except Exception as e:
34         raise ConnectionError(f"Gagal konek ke MySQL: {str(e)}")
35
36     # CREATE TABLE
37     try:
38         cursor.execute("""
39         CREATE TABLE IF NOT EXISTS `TXF_CUSTOMER_ADDRESS` (
40             id VARCHAR(255),
41             customer_id VARCHAR(255) not null,
42             address VARCHAR(255),
43             city VARCHAR(50),
44             province VARCHAR(25),
45             created_at DATETIME(3),
```



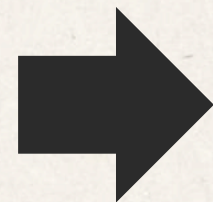
Library yang digunakan



Define path & connection



Setting parameter input_date & context['ds'] supaya job-nya bisa dijalankan manual (menggunakan json c/o: {"input_date":"2025-01-01"}) atau by scheduler



Raise exception error jika gagal terkoneksi ke database

PY Script 02: TXF_CUSTOMER_ADDRESS (CLEANSING & INJECT FROM TABLE STG_CUSTOMER_ADDRESS TO TABLE TXF_CUSTOMER_ADDRESS)

```
def ingest_txf_customer_address(**context):
    # CREATE TABLE
    try:
        cursor.execute("""
        CREATE TABLE IF NOT EXISTS `TXF_CUSTOMER_ADDRESS` (
            id VARCHAR(255),
            customer_id VARCHAR(255) not null,
            address VARCHAR(255),
            city VARCHAR(50),
            province VARCHAR(25),
            created_at DATETIME(3),
            LOAD_TIMESTAMP DATETIME(6) DEFAULT CURRENT_TIMESTAMP(6),
            PRIMARY KEY (customer_id)
        );
        """)

        # Duplicate handling
        cursor.execute("""
        DELETE FROM TXF_CUSTOMER_ADDRESS
        WHERE DATE(LOAD_TIMESTAMP) = %s
        """, (ds,))

        cursor.execute("""
        INSERT INTO `TXF_CUSTOMER_ADDRESS` (
            ID, CUSTOMER_ID, ADDRESS, CITY, PROVINCE, CREATED_AT
        )
        SELECT
            ID,
            CUSTOMER_ID,
            ADDRESS,
            CITY,
            PROVINCE,
            CAST(CREATED_AT AS DATETIME(3)) AS CREATED_AT
        FROM stg_customer_address
        ON DUPLICATE KEY UPDATE
            address = VALUES(address),
            city = VALUES(city),
            province = VALUES(province),
            created_at = VALUES(created_at),
            load_timestamp = CURRENT_TIMESTAMP(6);
        """)

        conn.commit()

        # time.sleep(2)

    except Exception as e:
        conn.rollback()
        raise RuntimeError(f"Gagal insert/update data: {e}")

    finally:
        cursor.close()
        conn.close()

    print(f"TXF_CUSTOMER_ADDRESS loaded for date {ds}")
```

DDL If not exist, menggunakan load_timestamp sbg control column

Menggunakan duplicate handling berdasarkan kontrol kolom load_timestamp, apabila dijalankan 2x dalam hari yang sama tidak akan duplicate

- Insert data

menggunakan type load update-insert (SCD Type 1) Karena tidak ada requirement untuk menyimpan history, jadi cukup update data lama dengan data baru jika terjadi duplikasi customer_id

Raise exception error rollback apabila gagal insert

```
# DAG
with DAG(
    dag_id='task_ingest_txf_customer_address',
    default_args=default_args,
    schedule='0 8 * * *', # jam 08:00
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=['dwh'],
) as dag:
    ingest_txf_customer_address()
```

Set dag id, tag & scheduler tiap hari jam 8 pagi

Task 02 Result in Mysql

Filter connections by name

mart_sales_datamart

16K

stg_after_sales_raw

128K

stg_customer_address

144K

stg_customers_raw

96K

stg_sales_raw

224K

txf_after_sales

80K

txf_customer_address

28K

Columns

AZ id (varchar(255))

AZ customer_id (varchar(255))

AZ address (varchar(255))

AZ city (varchar(50))

AZ province (varchar(25))

created_at (datetime(3))

LOAD_TIMESTAMP (datetime(6))

Show SQL

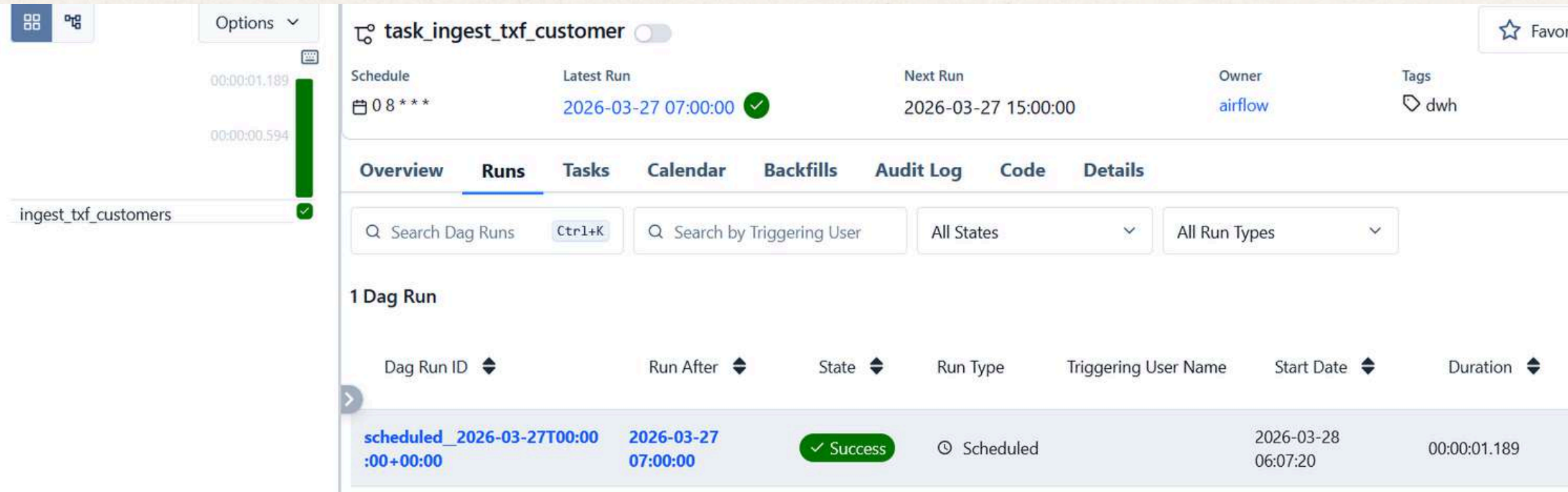
Enter a SQL expression to filter results (use Ctrl+Space)

	AZ id	AZ customer_id	AZ address	AZ city	AZ province	created_at	LOAD_TIMESTAMP
1	151	1	Jl. Cihampelas No. 0	Tanjungbalai	DI Yogyakarta	2026-03-02 09:57:49.818	2026-03-28 06:26:29.675171
2	163	10	Jalan Dipatiukur No. 567	Bogor	Papua	2026-03-10 10:35:08.689	2026-03-28 06:26:29.675171
3	897	100	Jl. Soekarno Hatta No. 590	Kota Administrasi Jakarta Pusat	Sumatera Selatan	2026-03-07 06:24:59.009	2026-03-28 06:26:29.675171
4	367	1000	Jl. S. Parman No. 342	Bontang	DKI Jakarta	2026-01-28 07:36:52.825	2026-03-28 06:26:29.675171
5	63	101	Jalan Kutai No. 12	Singkawang	Nusa Tenggara Timur	2026-02-22 13:35:08.810	2026-03-28 06:26:29.675171
6	325	102	Jl. Jamika No. 17	Jambi	Nusa Tenggara Timur	2026-01-03 02:19:41.254	2026-03-28 06:26:29.675171
7	169	103	Gg. M.H Thamrin No. 8	Malang	DKI Jakarta	2026-02-04 09:06:04.038	2026-03-28 06:26:29.675171
8	36	104	Jl. KH Amin Jasuta No. 11	Makassar	Kalimantan Tengah	2026-02-19 05:40:20.755	2026-03-28 06:26:29.675171
9	22	105	Gg. Rajiman No. 47	Kotamobagu	Papua Barat	2026-03-03 10:01:33.665	2026-03-28 06:26:29.675171
10	276	106	Jl. Veteran No. 87	Palembang	Sulawesi Barat	2026-01-29 06:56:49.425	2026-03-28 06:26:29.675171
11	546	107	Gang Sukabumi No. 63	Kota Administrasi Jakarta Pusat	Riau	2026-03-20 11:32:23.760	2026-03-28 06:26:29.675171
12	708	108	Gg. Moch. Toha No. 888	Semarang	Sulawesi Tenggara	2026-01-01 22:18:53.845	2026-03-28 06:26:29.675171

Task 02: Silver Layer/txf (task 2.a cleansing data)

2. Table customers

Table **stg_customers_raw** di cleansing dan diubah tipe datanya dari yang sebelumnya varchar jadi menyesuaikan dengan kolom yang ada (contoh date, int, dst) dan di-inject ke table table **txf_customers** dengan job airflow yang jalan daily tiap jam



PY Script 03: TXF_CUSTOMERS (CLEANSING & INJECT FROM TABLE STG_CUSTOMERS TO TABLE TXF_CUSTOMERS)

```
> task_ingest_txf_customer.py > ...
from airflow import DAG
from airflow.decorators import task
from airflow.providers.mysql.hooks.mysql import MySqlHook
from datetime import datetime, timedelta

MYSQL_CONN_ID = 'mysql-localhost'

default_args = {
    'owner': 'airflow',
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

@task
def ingest_txf_customers(**context):
    # Ambil tanggal Airflow (manual / scheduler)
    input_date = context.get("params", {}).get("input_date")
    ds = input_date if input_date else context['ds'] # YYYY-MM-DD

    try:
        mysql_hook = MySqlHook(mysql_conn_id=MYSQL_CONN_ID)
        conn = mysql_hook.get_conn()
        conn.ping()
        cursor = conn.cursor()
    except Exception as e:
        raise ConnectionError(f"Gagal konek ke MySQL: {str(e)}")

    try:
        # CREATE TABLE
        cursor.execute("""
        CREATE TABLE IF NOT EXISTS `TXF_CUSTOMERS` (
            ID VARCHAR(255) NOT NULL,
            NAME VARCHAR(255),
            DOB DATE,
            CREATED_AT DATETIME(6),
            LOAD_TIMESTAMP DATETIME(6) DEFAULT CURRENT_TIMESTAMP(6),
            PRIMARY KEY (ID)
        );
        """)

        # Duplicate handling
        cursor.execute("""
        DELETE FROM TXF_CUSTOMERS
        WHERE DATE(LOAD_TIMESTAMP) = %s
        """, (ds,))
```

Library yang digunakan

Define path & connection

Setting parameter input_date & context['ds'] supaya job-nya bisa dijalankan manual (menggunakan json c/o: {"input_date":"2025-01-01"}) atau by scheduler

Raise exception error jika gagal terkoneksi ke database

DDL If not exist, menggunakan load_timestamp sbg control column

Menggunakan duplicate handling berdasarkan kontrol kolom load_timestamp, apabila dijalankan 2x dalam hari yang sama tidak akan duplicate

PY Script 03: TXF_CUSTOMERS (CLEANSING & INJECT FROM TABLE STG_CUSTOMERS TO TABLE TXF_CUSTOMERS)

```
# SELECT dulu dari staging
cursor.execute("""
    INSERT INTO `TXF_CUSTOMERS` (id, name, dob, created_at)
    SELECT
        ID,
        NAME,
        CAST(
            CASE
                WHEN dob REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2}$' THEN dob
                WHEN dob REGEXP '^[0-9]{4}/[0-9]{2}/[0-9]{2}$' THEN STR_TO_DATE(dob, '%Y/%m/%d')
                WHEN dob REGEXP '^[0-9]{2}/[0-9]{2}/[0-9]{4}$' THEN STR_TO_DATE(dob, '%d/%m/%Y')
                ELSE NULL
            END AS DATE
        ),
        CAST(created_at AS DATETIME(3))
    FROM stg_customers_raw
    ON DUPLICATE KEY UPDATE
        NAME = VALUES(NAME),
        DOB = VALUES(DOB),
        CREATED_AT = VALUES(CREATED_AT),
        LOAD_TIMESTAMP = CURRENT_TIMESTAMP(6)
    """)

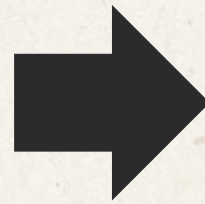
conn.commit()

except Exception as e:
    conn.rollback()
    raise RuntimeError(f"Gagal insert/update data: {e}")

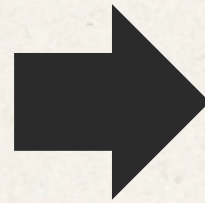
finally:
    cursor.close()
    conn.close()

print(f"TXF_CUSTOMERS loaded for date {ds}")

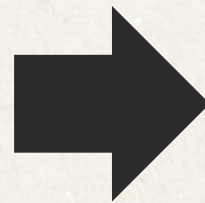
# DAG
with DAG(
    dag_id='task_ingest_txf_customer',
    default_args=default_args,
    # schedule='@daily',
    schedule='0 8 * * *',
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=['dwh'],
) as dag:
    ingest_txf_customers()
```



- Query cleaning & Insert data menggunakan type load update-insert (SCD Type 1) Karena tidak ada requirement untuk menyimpan history, jadi cukup update data lama dengan data baru jika terjadi duplikasi customer_id



Raise exception error rollback apabila gagal insert



Set dag id, tag & scheduler tiap hari jam 8 pagi

Task 02 Result in Mysql

Tables

mart_cust_service_prio_datamart

mart_sales_datamart 16K

stg_after_sales_raw 128K

stg_customer_address 144K

stg_customers_raw 96K

stg_sales_raw 224K

txf_after_sales 80K

txf_customer_address 128K

txf_customers 80K

Columns

A-Z ID (varchar(255))

A-Z NAME (varchar(255))

DOB (date)

CREATED_AT (datetime(6))

LOAD_TIMESTAMP (datetime(6))

Show SQL

Enter a SQL expression to filter results (use Ctrl+Space)

	A-Z ID	A-Z NAME	DOB	CREATED_AT	LOAD_TIMESTAMP
1	1	Antonio	1998-08-04	2025-03-01 14:24:40.012000	2026-03-28 06:07:20.877179
2	10	Julie	1980-09-12	2026-03-04 13:46:25.242000	2026-03-28 06:07:20.877179
3	100	Wang Inc	[NULL]	2026-01-30 17:50:26.136000	2026-03-28 06:07:20.877179
4	1000	Jon	2004-12-13	2026-01-01 18:55:58.582000	2026-03-28 06:07:20.877179
5	101	Catherine	[NULL]	2026-01-27 05:28:01.237000	2026-03-28 06:07:20.877179
6	102	Daniel	2008-07-11	2026-03-11 12:59:36.404000	2026-03-28 06:07:20.877179
7	103	Tina	2005-09-21	2026-01-04 06:36:47.485000	2026-03-28 06:07:20.877179
8	104	Manuel	1942-06-15	2026-03-08 16:16:59.407000	2026-03-28 06:07:20.877179
9	105	Jason	2006-01-14	2026-02-17 18:34:50.204000	2026-03-28 06:07:20.877179
10	106	Oscar	1952-02-10	2026-01-10 21:35:05.672000	2026-03-28 06:07:20.877179
11	107	Katherine	[NULL]	2026-02-02 01:20:37.182000	2026-03-28 06:07:20.877179
12	108	Raymond	1960-10-25	2026-03-16 13:36:38.983000	2026-03-28 06:07:20.877179
13	109	Deborah	1936-03-03	2026-01-02 09:48:36.112000	2026-03-28 06:07:20.877179

Task 02: Silver Layer/txf (task 2.a cleansing data)

3. Table sales

Table **stg_sales_raw** di cleansing dan diubah tipe datanya dari yang sebelumnya varchar jadi menyesuaikan dengan kolom yang ada (contoh date, int, dst) dan di-inject ke table table **txf_sales** dengan job airflow yang jalan daily tiap jam

Options

00:05:03

00:02:31

ingest_txf_sales

task_ingest_txf_sales

Schedule: 0 8 ***

Latest Run: 2026-03-27 23:29:38

Next Run: 2026-03-27 15:00:00

Owner: airflow

Tags: dwh

Overview

Runs

Tasks

Calendar

Backfills

Audit Log

Code

Details

Search Dag Runs

Search by Triggering User

All States

All Run Types

12 Dag Runs

Dag Run ID	Run After	State	Run Type	Triggering User Name	Start Date	Duration
manual_2026-03-27T16:29:38.443006+00:00	2026-03-27 23:29:38	Success	Manual	airflow	2026-03-27 23:29:38	00:00:02.635

PY Script 03: TXF_SALES (CLEANSING & INJECT FROM TABLE STG_SALES_RAW TO TABLE TXF_SALES)

```
from airflow import DAG
from airflow.decorators import task
from airflow.providers.mysql.hooks.mysql import MySqlHook
from datetime import datetime, timedelta

MYSQL_CONN_ID = 'mysql-localhost'

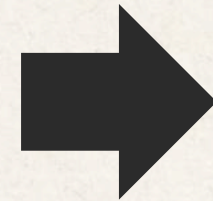
default_args = {
    'owner': 'airflow',
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

@task
def ingest_txf_sales(**context):
    # Ambil tanggal Airflow (manual / scheduler)
    input_date = context.get("params", {}).get("input_date")
    is_initial_load = context.get("params", {}).get("is_initial_load", False)
    ds = input_date if input_date else context['ds'] # YYYY-MM-DD
    ds_str = datetime.strptime(ds[:10], "%Y-%m-%d").strftime("%Y-%m-%d")

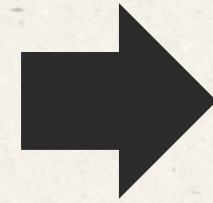
    # ds_str = datetime.strptime(ds, "%Y-%m-%d").strftime("%Y-%m-%d")

    try:
        mysql_hook = MySqlHook(mysql_conn_id=MYSQL_CONN_ID)
        conn = mysql_hook.get_conn()
        conn.ping()
        cursor = conn.cursor()
    except Exception as e:
        raise ConnectionError(f"Gagal konek ke MySQL: {str(e)}")

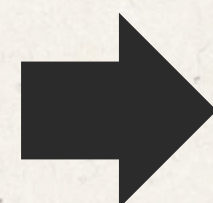
    try:
        # CREATE TABLE
        cursor.execute("""
        CREATE TABLE IF NOT EXISTS `TXF_SALES` (
            VIN VARCHAR(25) NOT NULL,
            CUSTOMER_ID VARCHAR(255),
            MODEL VARCHAR(50),
            INVOICE_DATE DATE NOT NULL,
            PRICE DECIMAL(38,5),
            CREATED_AT DATETIME(3),
            LOAD_TIMESTAMP DATETIME(6) DEFAULT CURRENT_TIMESTAMP(6),
            PRIMARY KEY (VIN, INVOICE_DATE),
            INDEX idx_invdate_customer (INVOICE_DATE, CUSTOMER_ID),
            INDEX idx_customer_id (CUSTOMER_ID),
            INDEX idx_invdate (INVOICE_DATE)
        )
        PARTITION BY RANGE (TO_DAYS(INVOICE_DATE)) (
            PARTITION p2022 VALUES LESS THAN (TO_DAYS('2023-01-01')),
            PARTITION p2023 VALUES LESS THAN (TO_DAYS('2024-01-01')),
            PARTITION p2024 VALUES LESS THAN (TO_DAYS('2025-01-01')),
            PARTITION p2025 VALUES LESS THAN (TO_DAYS('2026-01-01')),
            PARTITION pmax VALUES LESS THAN MAXVALUE
        );
        """)
```



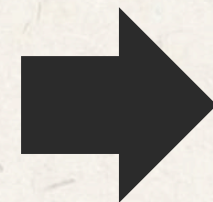
Library yang digunakan



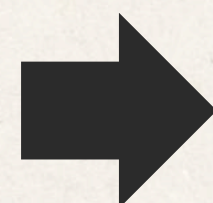
Define path & connection



Setting parameter input_date, initial_load & context['ds'] supaya job-nya bisa dijalankan manual (menggunakan json c/o: {"input_date":"2025-01-01"}) atau by scheduler , dan bisa menjalankan initial_load secara manual menggunakan json({"is_initial_load":true})



Raise exception error jika gagal terkoneksi ke database



DDL If not exist, menggunakan load_timestamp sbg control column

PY Script 03: TXF_SALES (CLEANSING & INJECT FROM TABLE STG_SALES_RAW TO TABLE TXF_SALES)

```
task_ingest_txf_sales.py > ...
def ingest_txf_sales(**context):
    # Duplicate handling
    if not is_initial_load:
        cursor.execute("""
            DELETE FROM TXF_SALES
            WHERE DATE(LOAD_TIMESTAMP) = %s
            """, (ds_str,))

    insert_sql_txf_sales = f"""
    INSERT INTO `TXF_SALES` (
        VIN, CUSTOMER_ID, MODEL, INVOICE_DATE, PRICE, CREATED_AT
    )
    SELECT
        VIN,
        CUSTOMER_ID,
        MODEL,
        CAST(INVOICE_DATE AS DATE) AS INVOICE_DATE,
        CAST(REPLACE(PRICE, '.', '' ) AS DECIMAL(38,5)) AS PRICE,
        CAST(CREATED_AT AS DATETIME(3)) AS CREATED_AT
    FROM stg_sales_raw
    {"WHERE DATE(CREATED_AT) = %s" if not is_initial_load else ""};
    """

    # Jika initial load di kolom json masukan contoh:
    # {
    #   "input_date": "2026-03-27",
    #   "is_initial_load": true
    # }

    params = (ds_str,) if not is_initial_load else None
    cursor.execute(insert_sql_txf_sales, params)

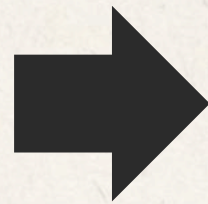
    conn.commit()

except Exception as e:
    conn.rollback()
    raise RuntimeError(f"Gagal insert/update data: {e}")

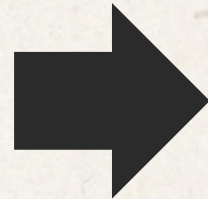
finally:
    cursor.close()
    conn.close()

print(f"TXF_SALES loaded for date {ds}")

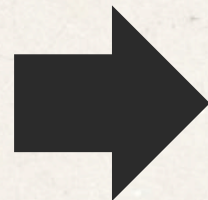
# DAG
with DAG(
    dag_id='task_ingest_txf_sales',
    default_args=default_args,
    # schedule='@daily',
    schedule='0 8 * * *',
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=['dwh'],
) as dag:
    ingest_txf_sales()
```



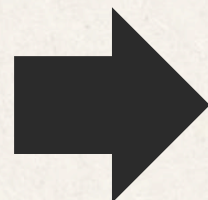
Menggunakan duplicate handling berdasarkan kontrol kolom load_timestamp, apabila dijalankan 2x dalam hari yang sama tidak akan duplicate



- Query cleaning & Insert data menggunakan type incremental load berdasarkan created_at(karena tanggalnya sama dengan invoice_date sbg as_of_date), initial load dijalankan manual diawal dengan menggunakan json {"is_initial_load":true}



Raise exception error rollback apabila gagal insert



Set dag id, tag & scheduler tiap hari jam 8 pagi

Task 02 Result in Mysql

mysql-localhost localhost:3306

Databases

mysql-dwh-dev

Tables

mart_cust_service_prio_datamart 128K

mart_sales_datamart 16K

stg_after_sales_raw 128K

stg_customer_address 144K

stg_customers_raw 96K

stg_sales_raw 224K

txf_after_sales 80K

txf_customer_address 128K

txf_customers 80K

txf_sales 80K

Columns

A-Z VIN (varchar(25))

A-Z CUSTOMER_ID (varchar(255))

A-Z MODEL (varchar(50))

INVOICE_DATE (date)

PRICE (decimal(38,5))

CREATED_AT (datetime(3))

LOAD_TIMESTAMP (datetime(6))

Show SQL

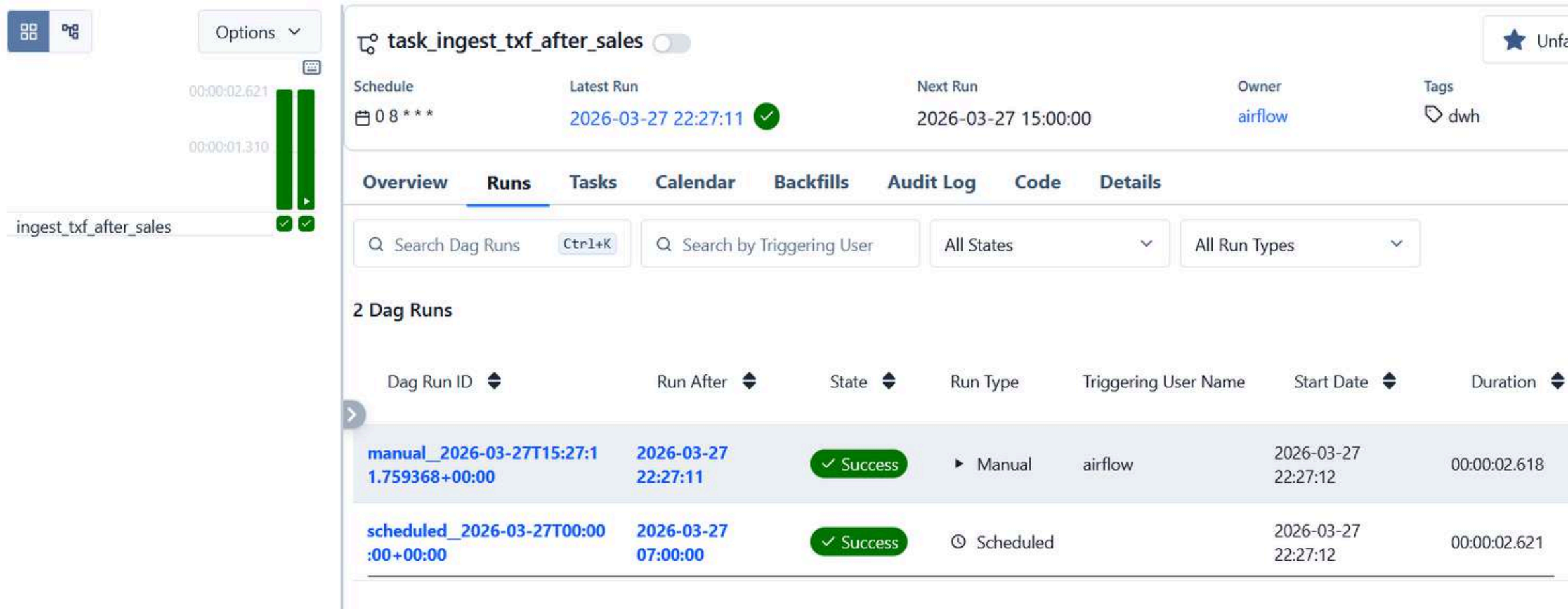
Enter a SQL expression to filter results (use Ctrl+Space)

	A-Z VIN	A-Z CUSTOMER_ID	A-Z MODEL	INVOICE_DATE	PRICE	CREATED_AT	LOAD_TIMESTAMP
1	00Q0ZO1FIU	248	RANGGO	2025-03-11	430,000,000	2025-03-11 17:11:43	2026-03-27 23:29:41.009217
2	00Q1T1HSKU	342	VELOS	2025-02-19	390,000,000	2025-02-19 02:39:11	2026-03-27 23:29:41.009217
3	0129H5OGWT	813	RAIZA	2025-03-15	350,000,000	2025-03-15 15:20:18	2026-03-27 23:29:41.009217
4	017R6NST81	991	VELOS	2025-01-24	390,000,000	2025-01-24 23:53:55	2026-03-27 23:29:41.009217
5	01EEU1D1A4	41	INNAVO	2025-03-10	600,000,000	2025-03-10 11:54:59	2026-03-27 23:29:41.009217
6	01RB27A5RZ	922	RAIZA	2025-03-24	350,000,000	2025-03-24 22:59:25	2026-03-27 23:29:41.009217
7	02DF4D0VB4	147	VELOS	2025-03-05	390,000,000	2025-03-05 15:26:51	2026-03-27 23:29:41.009217
8	03PIU17IRL	497	RANGGO	2025-01-15	430,000,000	2025-01-15 05:23:08	2026-03-27 23:29:41.009217
9	04EIBLK6TU	480	INNAVO	2025-03-04	600,000,000	2025-03-04 23:33:54	2026-03-27 23:29:41.009217
10	04ND0QHVIX	308	INNAVO	2025-01-22	600,000,000	2025-01-22 22:52:55	2026-03-27 23:29:41.009217
11	05WZ5TXFUX	569	RAIZA	2025-01-06	350,000,000	2025-01-06 22:01:08	2026-03-27 23:29:41.009217
12	0699IABLW6	991	RANGGO	2025-02-13	430,000,000	2025-02-13 01:18:16	2026-03-27 23:29:41.009217
13	08CL1OR6Q9	479	VELOS	2025-01-31	390,000,000	2025-01-31 15:44:28	2026-03-27 23:29:41.009217
14	0ALXM1NTAZ	856	INNAVO	2025-02-25	600,000,000	2025-02-25 09:53:09	2026-03-27 23:29:41.009217
15	0AR4Z57D0Q	114	RANGGO	2025-02-17	430,000,000	2025-02-17 08:21:59	2026-03-27 23:29:41.009217
16	0ASZA4BME4	302	INNAVO	2025-02-03	600,000,000	2025-02-03 19:33:36	2026-03-27 23:29:41.009217
17	0B3YKON3WK	597	VELOS	2025-03-23	390,000,000	2025-03-23 12:34:10	2026-03-27 23:29:41.009217
18	0BFS8EKAUX	404	INNAVO	2025-02-13	600,000,000	2025-02-13 09:50:15	2026-03-27 23:29:41.009217

Task 02: Silver Layer/txf (task 2.a cleansing data)

3. Table after_sales

Table **stg_after_sales_raw** di cleansing dan diubah tipe datanya dari yang sebelumnya varchar jadi menyesuaikan dengan kolom yang ada (contoh date, int, dst) dan di-inject ke table table **txf_after_sales** dengan job airflow yang jalan daily tiap jam



PY Script 03: TXF_AFTER_SALES (CLEANSING & INJECT FROM TABLE STG_AFTER_SALES_RAW TO TABLE TXF_AFTER_SALES)

```
from airflow import DAG
from airflow.decorators import task
from airflow.providers.mysql.hooks.mysql import MySqlHook
from datetime import datetime, timedelta

MYSQL_CONN_ID = 'mysql-localhost'

default_args = {
    'owner': 'airflow',
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

@task
def ingest_txf_after_sales(**context):
    # Ambil tanggal Airflow (manual / scheduler)
    input_date = context.get("params", {}).get("input_date")
    is_initial_load = context.get("params", {}).get("is_initial_load", False)
    ds = input_date if input_date else context['ds'] # YYYY-MM-DD
    ds_str = datetime.strptime(ds[:10], "%Y-%m-%d").strftime("%Y-%m-%d")

    try:
        mysql_hook = MySqlHook(mysql_conn_id=MYSQL_CONN_ID)
        conn = mysql_hook.get_conn()
        conn.ping()
        cursor = conn.cursor()

    except Exception as e:
        raise ConnectionError(f"Gagal konek ke MySQL: {str(e)}")

    # CREATE TABLE
    try:
        cursor.execute("""
        CREATE TABLE IF NOT EXISTS `TXF_AFTER_SALES` (
            service_ticket VARCHAR(10),
            vin VARCHAR(50),
            customer_id VARCHAR(255),
            model VARCHAR(50),
            service_date DATE,
            service_type VARCHAR(5),
            created_at DATETIME(3),
            LOAD_TIMESTAMP DATETIME(6) DEFAULT CURRENT_TIMESTAMP(6),
            PRIMARY KEY (SERVICE_TICKET)
        );
        """)

    # Duplicate handling
    if not is_initial_load:
        cursor.execute("""
        DELETE FROM TXF_SALES
        WHERE DATE(LOAD_TIMESTAMP) = %s
        """, (ds_str,))
```

Library yang digunakan

Define path & connection

Setting parameter input_date, initial_load & context['ds'] supaya job-nya bisa dijalankan manual (menggunakan json c/o: {"input_date":"2025-01-01"}) atau by scheduler, dan bisa menjalankan initial_load secara manual menggunakan json({"is_initial_load":true})

Raise exception error jika gagal terkoneksi ke database

DDL If not exist, menggunakan load_timestamp sbg control column

Menggunakan duplicate handling berdasarkan kontrol kolom load_timestamp, apabila dijalankan 2x dalam hari yang sama tidak akan duplicate

PY Script 03: TXF_AFTER_SALES (CLEANSING & INJECT FROM TABLE STG_AFTER_SALES_RAW TO TABLE TXF_AFTER_SALES)

```
insert_sql_txf_sales = f"""
INSERT INTO `TXF_AFTER_SALES` (
    SERVICE_TICKET, VIN, CUSTOMER_ID, MODEL, SERVICE_DATE, SERVICE_TYPE, CREATED_AT
)
SELECT
    SERVICE_TICKET,
    VIN,
    CUSTOMER_ID,
    MODEL,
    CAST(SERVICE_DATE AS DATE) AS SERVICE_DATE,
    SERVICE_TYPE,
    CAST(CREATED_AT AS DATETIME(3)) AS CREATED_AT
FROM stg_after_sales_raw
{"WHERE DATE(CREATED_AT) = %s" if not is_initial_load else ""};
"""

params = (ds_str,) if not is_initial_load else None
cursor.execute(insert_sql_txf_sales, params)

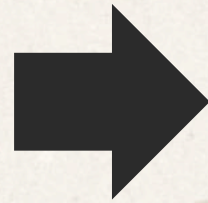
conn.commit()

except Exception as e:
    conn.rollback()
    raise RuntimeError(f"Gagal insert/update data: {e}")

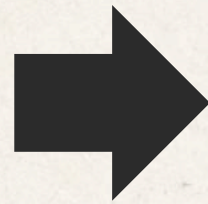
finally:
    cursor.close()
    conn.close()

print(f"TXF_AFTER_SALES loaded for date {ds}")

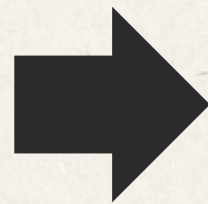
# DAG
with DAG(
    dag_id='task_ingest_txf_after_sales',
    default_args=default_args,
    # schedule='@daily',
    schedule='0 8 * * *',
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=['dwh'],
) as dag:
    ingest_txf_after_sales()
```



- Query cleaning & Insert data menggunakan type incremental load berdasarkan created_at(karena tanggalnya sama dengan invoice_date sbg as_of_date), initial load dijalankan manual diawal dengan menggunakan json {"is_initial_load":true}



Raise exception error rollback apabila gagal insert



Set dag id, tag & scheduler tiap hari jam 8 pagi

Task 02 Result in Mysql

dwh-dev-123 dwh-dev-123.database.windows.net:1433

mysql-localhost localhost:3306

Databases

mysql-dwh-dev

Tables

mart_cust_service_prio_datamart128K

mart_sales_datamart16K

stg_after_sales_raw128K

stg_customer_address144K

stg_customers_raw96K

stg_sales_raw224K

txf_after_sales80K

Columns

AZ service_ticket (varchar(10))

AZ vin (varchar(50))

AZ customer_id (varchar(255))

AZ model (varchar(50))

service_date (date)

AZ service_type (varchar(5))

created_at (datetime(3))

LOAD_TIMESTAMP (datetime(6))

Show SQL

Enter a SQL expression to filter results (use Ctrl+Space)

Grid

Text

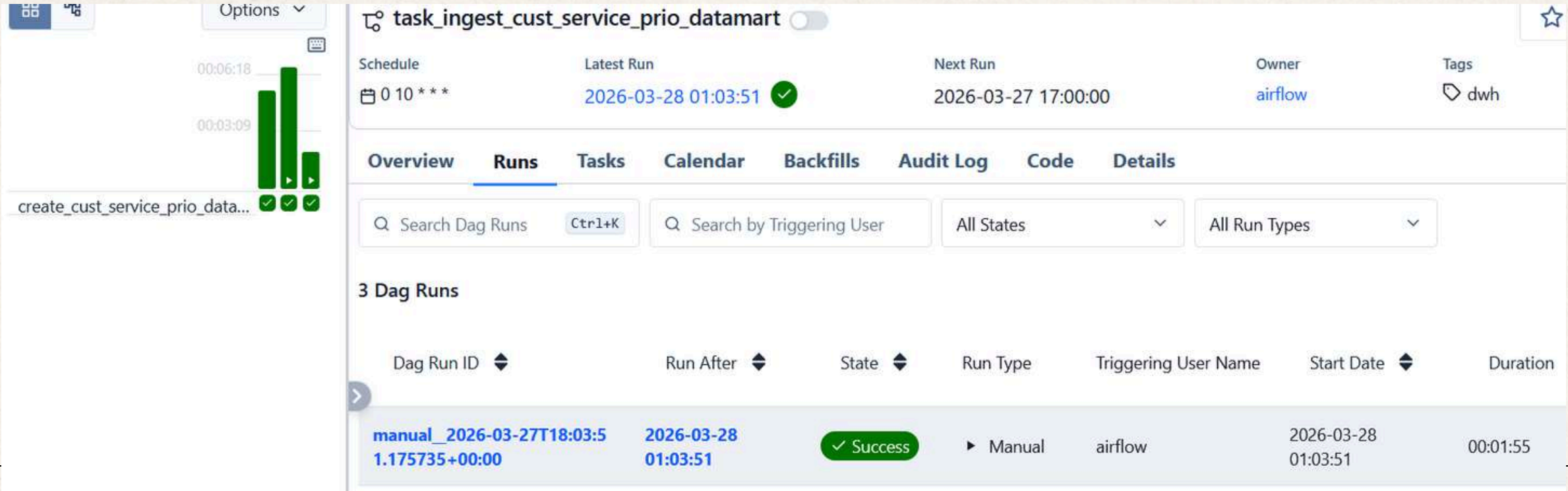
	AZ service_ticket	AZ vin	AZ customer_id	AZ model	service_date	AZ service_type	created_at	LOAD_TIMESTAMP
1	T103-aem4	J88GVQUUX3	143	VELOS	2025-10-19	BP	2025-10-19 21:01:28.240	2026-03-27 22:27:0
2	T105-fez8	I4I7X7JNQ9	810	INNAVO	2026-05-23	PM	2026-05-23 13:38:59.065	2026-03-27 22:27:0
3	T105-uwo8	RYI2DSX37B	494	RAIZA	2025-11-30	BP	2025-11-30 15:09:36.345	2026-03-27 22:27:0
4	T108-giv9	GKL4JPPSBA	5	RANGGO	2025-09-02	GR	2025-09-02 07:40:42.678	2026-03-27 22:27:0
5	T108-zda4	W99VFFB2RN	17	VELOS	2026-06-10	BP	2026-06-10 18:40:55.794	2026-03-27 22:27:0
6	T109-ziq4	JDQIADZVMG	318	RAIZA	2025-08-08	BP	2025-08-08 07:14:50.384	2026-03-27 22:27:0
7	T112-cyu6	DRJVAP0ATD	949	RANGGO	2026-11-05	BP	2026-11-05 22:20:20.999	2026-03-27 22:27:0
8	T112-ebw8	HIRAC0VGyQ	875	RANGGO	2025-03-03	GR	2025-03-03 22:46:17.097	2026-03-27 22:27:0
9	T113-abm1	A0G78U1E97	70	RAIZA	2025-10-22	BP	2025-10-22 16:04:37.390	2026-03-27 22:27:0
10	T113-rgk5	GDDNMAC5Q6	599	VELOS	2025-07-03	PM	2025-07-03 20:33:45.328	2026-03-27 22:27:0
11	T115-fen8	3XOSN7CHWS	773	VELOS	2026-12-01	GR	2026-12-01 16:31:41.423	2026-03-27 22:27:0
12	T115-ukb9	GSHHWNJPPH	612	VELOS	2026-11-20	GR	2026-11-20 12:53:15.564	2026-03-27 22:27:0
13	T115-wjv3	41X6XJRZ5E	767	RAIZA	2026-06-05	PM	2026-06-05 21:30:56.687	2026-03-27 22:27:0
14	T116-olt9	5G5P6262V9	53	INNAVO	2025-10-05	GR	2025-10-05 09:18:22.463	2026-03-27 22:27:0
15	T116-rkh3	92VN7XV9I3	933	INNAVO	2026-05-06	PM	2026-05-06 17:53:32.355	2026-03-27 22:27:0
16	T117-lzg8	QK838I8F06	751	INNAVO	2026-04-13	PM	2026-04-13 17:54:30.991	2026-03-27 22:27:0

Task 02: Gold Layer/datamart (task 2.b)

1. Table mart_cust_service_prio_datamart

Table mart_cust_service_prio_datamart didapat dari query aggregate dari table txf_after_sales, datamart ini berisi data berapa kali kendaraan customer diservice beserta prioritasnya, dengan ekspektasi hasil sbg berikut:

periode	vin	customer_name	address	count_service	priority
format YYYY				jumlah berapa kali melakukan servis	HIGH: servis >10x MED: servis 5-10x LOW: servis <5x



PY Script 04: Table mart_cust_service_prio_datamart

```
import os
from airflow import DAG
from airflow.decorators import task
from datetime import datetime, timedelta

from spark_mysql_utils import (
    create_spark_session,
    get_mysql_jdbc_config,
    prepare_target_table,
    uppercase_columns,
)

MYSQL_CONN_ID = os.getenv('MYSQL_CONN_ID', 'mysql-localhost')
TARGET_TABLE = 'MART_SALES_DATAMART'

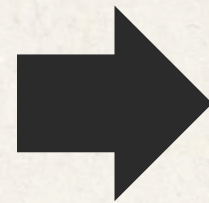
default_args = {
    'owner': 'airflow',
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

@task
def create_sales_datamart():
    from pyspark.sql import functions as F
    from pyspark.sql.types import DecimalType

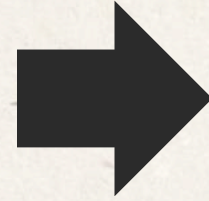
    mysql_hook, jdbc_url, jdbc_properties = get_mysql_jdbc_config(MYSQL_CONN_ID)
    spark = create_spark_session('sales-datamart')

    try:
        sales_df = uppercase_columns(
            spark.read.jdbc(
                url=jdbc_url,
                table='TXF_SALES',
                properties=jdbc_properties,
            )
        )

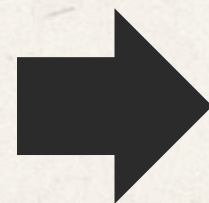
        price = F.col('PRICE')
        sales_datamart_df = (
            sales_df
            .withColumn('PERIODE', F.date_format(F.col('INVOICE_DATE'), 'yyyy-MM'))
            .withColumn(
                'CLASS',
                F.when(price.between(100000000, 250000000), F.lit('LOW'))
                .when(price.between(250000001, 400000000), F.lit('MEDIUM'))
                .when(price > 400000000, F.lit('HIGH')),
            )
            .groupBy('PERIODE', 'CLASS', 'MODEL')
            .agg(
                F.sum('PRICE')
                .cast(DecimalType(38, 5))
                .alias('TOTAL')
            )
        )
```



Library yang digunakan



Define connection



Query datamart & Insert data
menggunakan spark dengan type load truncate insert

PY Script 04: Table mart_cust_service_prio_datamart

```
.select('PERIODE', 'CLASS', 'MODEL', 'TOTAL')
)

prepare_target_table(mysql_hook, """
CREATE TABLE IF NOT EXISTS `MART_SALES_DATAMART` (
  PERIODE VARCHAR(7) NOT NULL,
  CLASS VARCHAR(10),
  MODEL VARCHAR(50),
  TOTAL DECIMAL(38,5),
  LOAD_TIMESTAMP DATETIME(6) DEFAULT CURRENT_TIMESTAMP(6),
  PRIMARY KEY (PERIODE, CLASS, MODEL)
);
""", TARGET_TABLE)

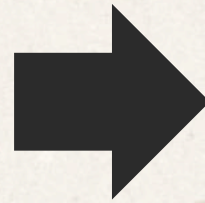
(
  sales_datamart_df.write
  .mode('append')
  .option('batchsize', os.getenv('SPARK_JDBC_BATCH_SIZE', '1000'))
  .jdbc(
    url=jdbc_url,
    table=TARGET_TABLE,
    properties=jdbc_properties,
  )
)

print(f"{TARGET_TABLE} successfully created and loaded with PySpark")

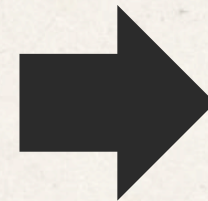
except Exception as e:
  raise RuntimeError(f"Gagal membuat datamart: {e}")

finally:
  spark.stop()

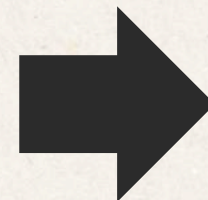
# DAG
with DAG(
  dag_id='task_ingest_sales_datamart',
  default_args=default_args,
  # schedule='@daily',
  schedule='0 10 * * *',
  start_date=datetime(2026, 1, 1),
  catchup=False,
  tags=['dwh'],
) as dag:
  create_sales_datamart()
```



Query datamart & Insert data
menggunakan spark dengan type load truncate insert



Raise exception error rollback apabila gagal membuat
table



Set dag id, tag & scheduler tiap hari jam 10 pagi

Task 02 Result in Mysql

Databases

mysql-dwh-dev

Tables

mart_cust_service_prio_datamart

28K

mart_sales_datamart

16K

stg_after_sales_raw

128K

stg_customer_address

144K

stg_customers_raw

96K

stg_sales_raw

224K

txf_after_sales

80K

txf_customer_address

128K

txf_customers

80K

txf_sales

80K

Views

Grid

Text

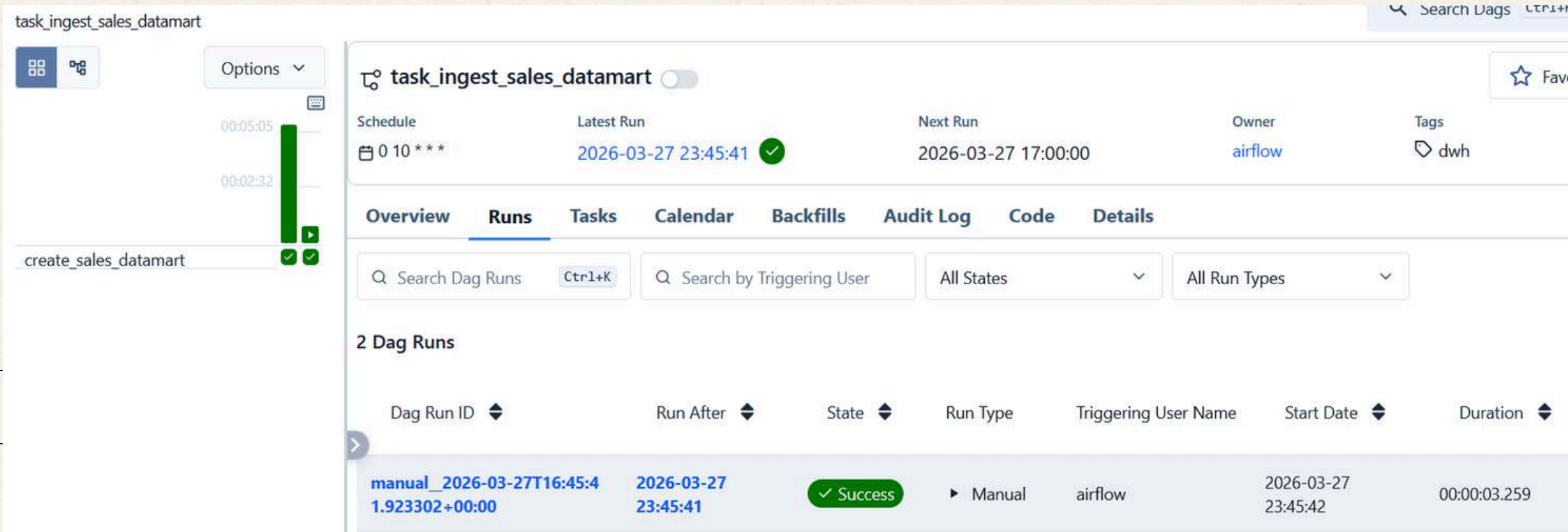
	123 periode	A-Z vin	A-Z customer_name	A-Z address	123 count_service	A-Z priority	LOAD_TIMESTAMP
1	2,025	J88GVQUUX3	John	Jalan Veteran No. 042	1	LOW	2026-03-28 01:08:37.443968
2	2,026	I4I7X7JNQ9	Fleming, Gutierrez and Walker	Gang Sukajadi No. 6	1	LOW	2026-03-28 01:08:37.443968
3	2,025	RYI2DSX37B	Dickson Ltd	Gang Kutisari Selatan No. 19	1	LOW	2026-03-28 01:08:37.443968
4	2,025	GKL4JPPSBA	Erik	Jalan Ronggowarsito No. 69	1	LOW	2026-03-28 01:08:37.443968
5	2,026	W99VFFB2RN	Kelli	Jalan PHH. Mustofa No. 5	1	LOW	2026-03-28 01:08:37.443968
6	2,025	JDQIADZVMG	Thomas	Jalan Raya Setiabudhi No. 131	1	LOW	2026-03-28 01:08:37.443968
7	2,026	DRJVAP0ATD	Cody	Gang Raya Setiabudhi No. 158	1	LOW	2026-03-28 01:08:37.443968
8	2,025	HIRAC0VGYQ	Susan	Jalan Gedebage Selatan No. 55	1	LOW	2026-03-28 01:08:37.443968
9	2,025	A0G78U1E97	Glass Inc	Gg. Peta No. 505	1	LOW	2026-03-28 01:08:37.443968
10	2,025	GDDNMAC5Q6	Robert	Jl. Raya Setiabudhi No. 431	1	LOW	2026-03-28 01:08:37.443968
11	2,026	3XOSN7CHWS	Rebecca	Jalan K.H. Wahid Hasyim No. 2	1	LOW	2026-03-28 01:08:37.443968

Task 02: Gold Layer/datamart (task 2.b)

2. Table mart_sales_datamart

Table mart_sales_datamart didapat dari query aggregate dari table txf_customers, txf_customer_address & txf_sales. Table ini berisi periode sales, range harga kendaraan yang terjual. tipe model kendaraan, dan total nominal penjualan per periode sales dengan ekspektasi hasil sbg berikut:

periode	class	model	total
format YYYY-MM	LOW: range 100-250juta MEDIUM: range 250-400juta HIGH: >400juta		jumlah nominal penjualan



PY Script 05: Table mart_sales_datamart

```
import os
from airflow import DAG
from airflow.decorators import task
from datetime import datetime, timedelta

from spark_mysql_utils import (
    create_spark_session,
    get_mysql_jdbc_config,
    prepare_target_table,
    uppercase_columns,
)

MYSQL_CONN_ID = os.getenv('MYSQL_CONN_ID', 'mysql-localhost')
TARGET_TABLE = 'MART_SALES_DATAMART'

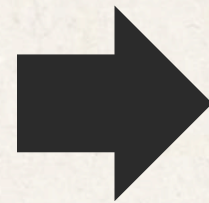
default_args = {
    'owner': 'airflow',
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

@task
def create_sales_datamart():
    from pyspark.sql import functions as F
    from pyspark.sql.types import DecimalType

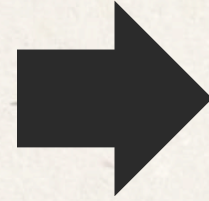
    mysql_hook, jdbc_url, jdbc_properties = get_mysql_jdbc_config(MYSQL_CONN_ID)
    spark = create_spark_session('sales-datamart')

    try:
        sales_df = uppercase_columns(
            spark.read.jdbc(
                url=jdbc_url,
                table='TXF_SALES',
                properties=jdbc_properties,
            )
        )

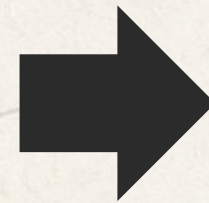
        price = F.col('PRICE')
        sales_datamart_df = (
            sales_df
            .withColumn('PERIODE', F.date_format(F.col('INVOICE_DATE'), 'yyyy-MM'))
            .withColumn(
                'CLASS',
                F.when(price.between(100000000, 250000000), F.lit('LOW'))
                .when(price.between(250000001, 400000000), F.lit('MEDIUM'))
                .when(price > 400000000, F.lit('HIGH')),
            )
            .groupBy('PERIODE', 'CLASS', 'MODEL')
            .agg(
                F.sum('PRICE')
                .cast(DecimalType(38, 5))
                .alias('TOTAL')
            )
        )
```



Library yang digunakan



Define connection



Query datamart & Insert data
menggunakan spark dan type load truncate insert

PY Script 05: Table mart_sales_datamart

```
        .select('PERIODE', 'CLASS', 'MODEL', 'TOTAL')
    )

    prepare_target_table(mysql_hook, """
    CREATE TABLE IF NOT EXISTS `MART_SALES_DATAMART` (
        PERIODE VARCHAR(7) NOT NULL,
        CLASS VARCHAR(10),
        MODEL VARCHAR(50),
        TOTAL DECIMAL(38,5),
        LOAD_TIMESTAMP DATETIME(6) DEFAULT CURRENT_TIMESTAMP(6),
        PRIMARY KEY (PERIODE, CLASS, MODEL)
    );
    """, TARGET_TABLE)

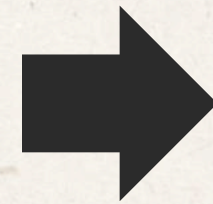
    (
        sales_datamart_df.write
        .mode('append')
        .option('batchsize', os.getenv('SPARK_JDBC_BATCH_SIZE', '1000'))
        .jdbc(
            url=jdbc_url,
            table=TARGET_TABLE,
            properties=jdbc_properties,
        )
    )

    print(f"{TARGET_TABLE} successfully created and loaded with PySpark")

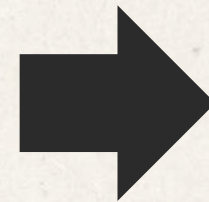
except Exception as e:
    raise RuntimeError(f"Gagal membuat datamart: {e}")

finally:
    spark.stop()

# DAG
with DAG(
    dag_id='task_ingest_sales_datamart',
    default_args=default_args,
    # schedule='@daily',
    schedule='0 10 * * *',
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=['dwh'],
) as dag:
    create_sales_datamart()
```



Raise exception error rollback apabila gagal create table



Set dag id, tag & scheduler tiap hari jam 10 pagi

Task 02 Result in Mysql

Databases

mysql-dwh-dev

Tables

mart_cust_service_prio_datamart

128K

mart_sales_datamart

16K

Columns

A-Z PERIODE (varchar(7))

A-Z CLASS (varchar(10))

A-Z MODEL (varchar(50))

123 TOTAL (decimal(38,5))

LOAD_TIMESTAMP (datetime(6))

Constraints

Foreign Keys

References

Triggers

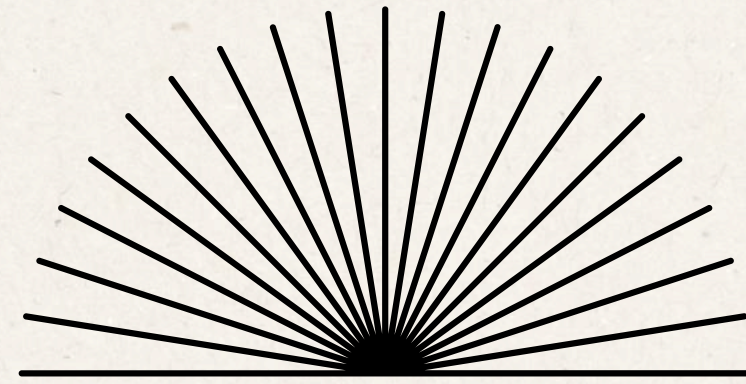
Indexes

Partitions

Grid

Text

	A-Z PERIODE	A-Z CLASS	A-Z MODEL	123 TOTAL	LOAD_TIMESTAMP
1	2025-01	HIGH	INNAVO	84,000,000,000	2026-03-27 23:50:47.695494
2	2025-01	HIGH	RANGGO	52,460,000,000	2026-03-27 23:50:47.695494
3	2025-01	MEDIUM	RAIZA	47,600,000,000	2026-03-27 23:50:47.695494
4	2025-01	MEDIUM	VELOS	51,480,000,000	2026-03-27 23:50:47.695494
5	2025-02	HIGH	INNAVO	152,400,000,000	2026-03-27 23:50:47.695494
6	2025-02	HIGH	RANGGO	93,310,000,000	2026-03-27 23:50:47.695494
7	2025-02	MEDIUM	RAIZA	76,300,000,000	2026-03-27 23:50:47.695494
8	2025-02	MEDIUM	VELOS	87,750,000,000	2026-03-27 23:50:47.695494
9	2025-03	HIGH	INNAVO	88,200,000,000	2026-03-27 23:50:47.695494
10	2025-03	HIGH	RANGGO	53,750,000,000	2026-03-27 23:50:47.695494
11	2025-03	MEDIUM	RAIZA	55,650,000,000	2026-03-27 23:50:47.695494
12	2025-03	MEDIUM	VELOS	48,360,000,000	2026-03-27 23:50:47.695494
13	2025-04	MEDIUM	RAIZA	350,000,000	2026-03-27 23:50:47.695494



Thank you